tdb — Documentation

Interface graphique pour Tarantool

Table des matières

Démarrage rapide — tdb **1**
- Prérequis . . . 2
- Lancement . . . 2
- Connexion . . . 2
- Présentation de l'interface . . . 3
- Créer les espaces (tables) . . . 3
  - 1 — Tables de référence . . . 4
  - 2 — Tables principales . . . 4
  - 3 — Tables de jointure many-to-many . . . 5
  - 4 — Table de transaction . . . 5
- Ajouter et gérer des champs . . . 5
  - Créer un champ simple . . . 5
  - Exemple : champs de l'espace `auteurs` . . . 6
  - Exemple : champs de l'espace `livres` . . . 6
  - Colonne calculée (λ) . . . 6
  - Trigger formula ((trigger)) . . . 7
  - Modifier un champ existant . . . 7
- Saisir et éditer des données . . . 7
- Relations entre espaces . . . 8
- Vues personnalisées (YAML) . . . 8
  - Créer une vue . . . 8
  - Éditeur YAML avec schéma ERD . . . 9
  - Vue « Gestion des prêts » . . . 11
  - Widget agrégat — Emprunts par genre . . . 12
- Gestion des utilisateurs et droits (admin) . . . 12
  - Utilisateurs . . . 13
  - Groupes . . . 13
  - Export et import de snapshots . . . 14
- Profil utilisateur . . . 14
- Renommer ou supprimer un espace . . . 15
- Raccourcis clavier . . . 15

Démarrage rapide — tdb

tdb est inspiré de Grist — pour tout usage sérieux ou en production, l'utilisateur est invité à se tourner vers ce dernier en priorité.

tdb est une interface graphique pour Tarantool permettant de créer et gérer des espaces de données (tables), de les relier entre eux, d'y définir des colonnes calculées ou des triggers, et de composer des vues personnalisées déclaratives (YAML).

Ce guide vous emmène pas à pas de zéro à une application de **gestion de bibliothèque scolaire** fonctionnelle : catalogue de livres, exemplaires physiques, emprunteurs et prêts.

Prérequis

- Docker (ou Tarantool 3.x installé localement)
- `make`, `moonc` (compilateur MoonScript), `coffee` (CoffeeScript) — uniquement pour modifier les sources

---

Lancement

```
git clone <repo>
cd tdb
docker compose up
```

L'interface est disponible sur **http ://localhost :8080**.

Connexion

FIGURE 1 – Écran de connexion

Au premier démarrage, un compte **admin** est créé automatiquement avec le mot de passe **admin**.

> [!] Un bandeau d'avertissement s'affiche tant que ce mot de passe par défaut n'a pas été changé. Cliquez sur « Changer maintenant » ou passez par le menu profil (en bas à gauche).

Présentation de l'interface

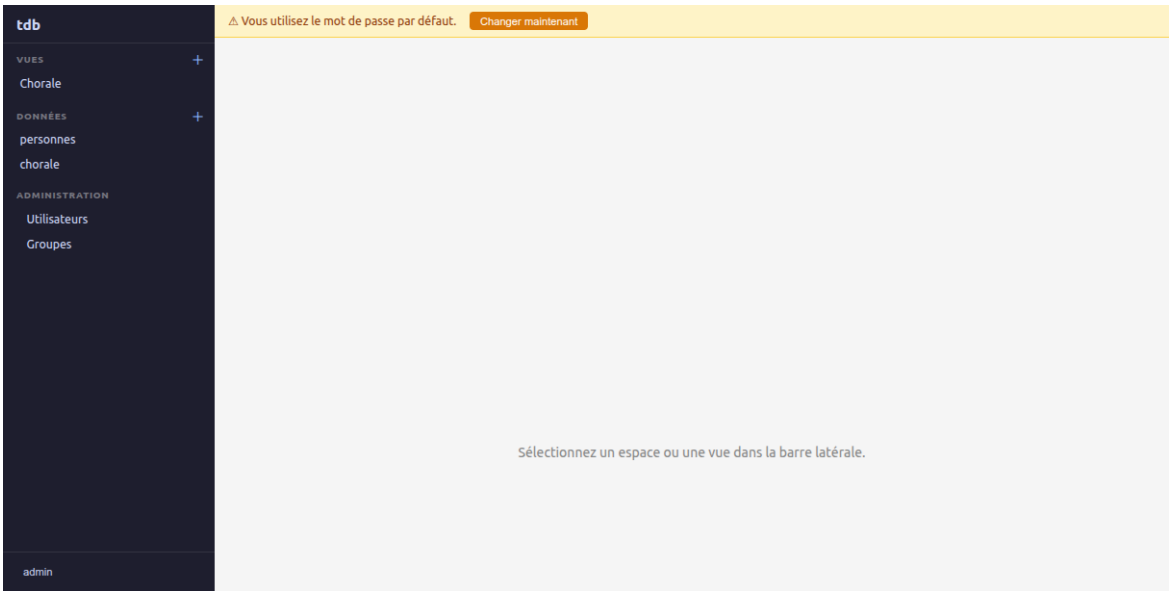

FIGURE 2 – Interface principale

L'interface est divisée en deux zones :

Zone	Rôle
Barre latérale (gauche)	Navigation entre Vues, Données et Administration
Contenu (droite)	Grille de données, éditeur de vue ou panel d'administration

---

Créer les espaces (tables)

La bibliothèque repose sur 12 tables. Créez-les dans l'ordre suivant pour faciliter la définition des relations : d'abord les tables de référence, puis les tables principales, puis les tables de jointure, et enfin la table de transaction.

1 — Tables de référence

Ces tables n'ont pas de dépendance vers d'autres tables.

1. Dans la section **Données** de la barre latérale, cliquez sur **+**.
2. Saisissez le nom de l'espace et validez.
3. Répétez pour chaque table.

Espace	Rôle
`genres`	Genres littéraires (roman, BD, documentaire…)
`auteurs`	Auteurs des ouvrages
`classes`	Classes scolaires de l'établissement
`motscles`	Mots-clés thématiques
`etiquettes`	Étiquettes de classement (prix, coups de cœur…)

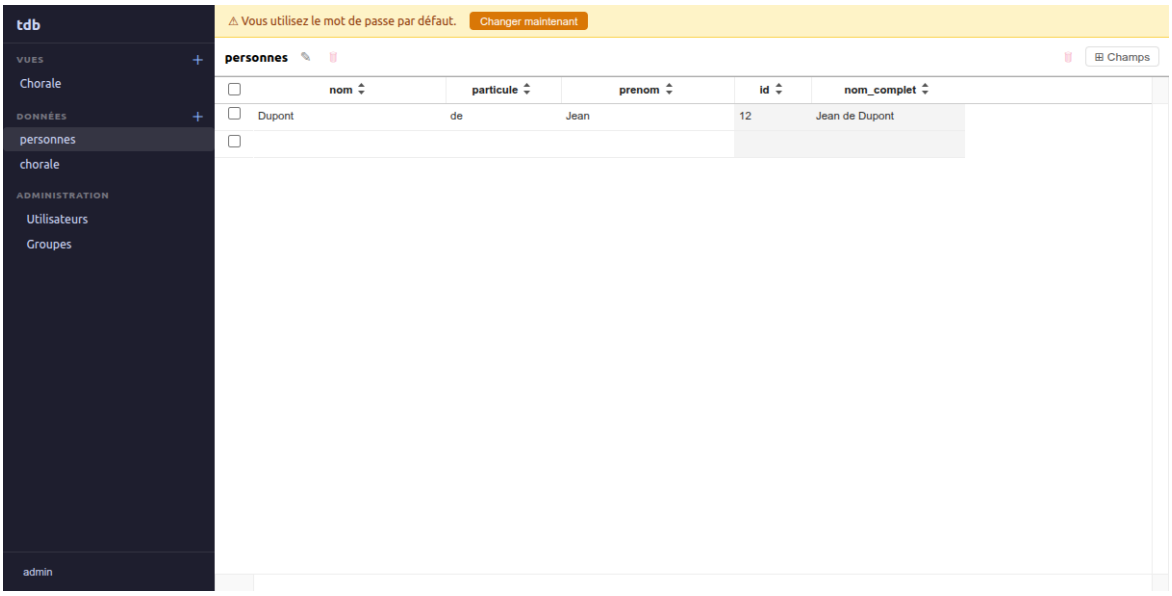

FIGURE 3 – Espace de données avec grille

2 — Tables principales

Espace	Rôle
`livres`	Catalogue des titres (références bibliographiques)
`personnes`	Élèves et personnels pouvant emprunter
`exemplaires`	Copies physiques de chaque titre
`coordonnees`	Coordonnées de contact des personnes

3 — Tables de jointure many-to-many

Espace	Rôle
`livres_motscles`	Association livre ↔ mot-clé
`livres_etiquettes`	Association livre ↔ étiquette

4 — Table de transaction

Espace	Rôle
`emprunts`	Enregistrement des prêts (exemplaire + emprunteur + dates)

Ajouter et gérer des champs

Cliquez sur **[#] Champs** dans la barre d'outils pour ouvrir le panel latéral.

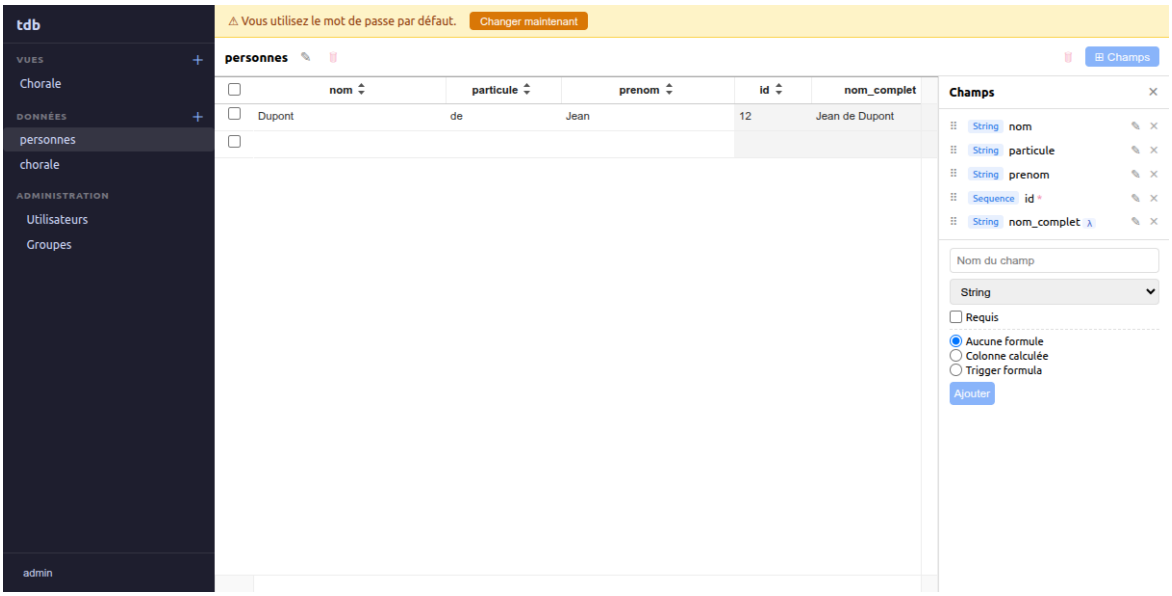

FIGURE 4 – Panel de champs

Créer un champ simple

1. Saisissez le **nom** du champ.
2. Choisissez le **type** : `String`, `Int`, `Float`, `Boolean`, `UUID`, `Séquence`, `Any`, `Map`, `Array`, ou `Relation`.
3. Cochez **Requis** si la valeur ne peut pas être nulle.
4. Cliquez sur **Ajouter**.

Exemple : champs de l'espace `auteurs`

Champ	Type	Requis
`id`	Séquence	—
`particule`	String	non
`nom`	String	**oui**
`prenom`	String	non

Exemple : champs de l'espace `livres`

Champ	Type	Requis	Remarque
`id`	Séquence	—	
`titre`	String	**oui**	
`auteur_id`	Relation	non	→ `auteurs`
`genre_id`	Relation	non	→ `genres`
`cote`	String	non	Cote de rangement
`isbn`	String	non	Nullable
`annee`	Int	non	Année de parution, nullable

Colonne calculée (λ)

Sélectionnez **Colonne calculée** et saisissez une expression MoonScript ou Lua. La formule est le corps d'une fonction `(self, space) -> <formule>`; en MoonScript, `@champ` accède à `self.champ`. La valeur est recalculée à chaque lecture ; elle n'est pas stockée.

Exemple : **`nom_complet` dans `auteurs`** — combine prénom, particule et nom en tenant compte des valeurs nulles :

```
parts = {}
table.insert(parts, @prenom) if @prenom and @prenom != ""
table.insert(parts, @particule) if @particule and @particule != ""
table.insert(parts, @nom) if @nom and @nom != ""
table.concat(parts, " ")
```

Le paramètre `space` donne accès aux enregistrements d'un autre espace (full scan) :

```
-- Compter les exemplaires disponibles pour ce livre
n = 0
for e in *space("exemplaires")
  n += 1 if e.livre_id == @id and e.disponible
n
```

Trigger formula ((trigger))

Sélectionnez **Trigger formula** et, optionnellement, listez les champs déclencheurs. La formule est exécutée et le résultat **stocké** lors de chaque création ou modification des champs listés.

Exemple : **`cote_auto` dans `livres`** — génère une cote bibliographique automatique à partir des trois premières lettres du titre et de l'année (déclenché sur `titre` et `annee`) :

```
-- triggerFields: ["titre", "annee"]
prefix = (@titre or "")\upper!\sub(1, 3)\gsub("[^%A]", "")
annee  = if @annee then tostring(@annee) else "????"
"#{prefix}-#{annee}"
```

Modifier un champ existant

Cliquez sur **(crayon)** à côté du champ pour éditer son nom, type, formule ou trigger.

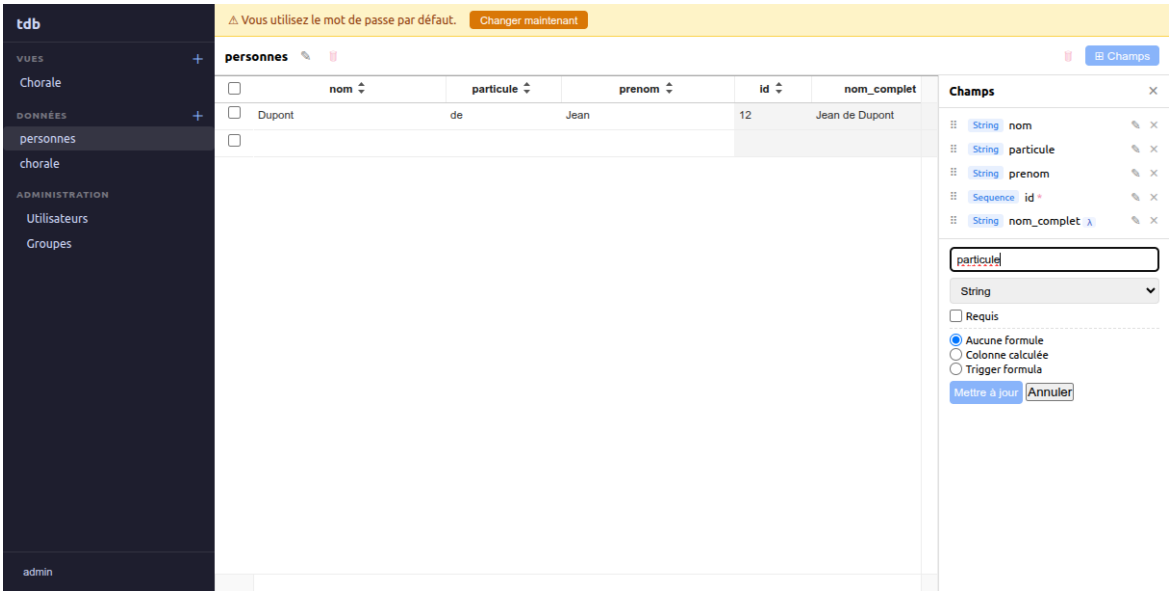

FIGURE 5 – Édition d'un champ

Saisir et éditer des données

- Cliquez sur une cellule pour l'éditer directement dans la grille.
- Appuyez sur **Entrée** ou tabulation pour passer à la suivante.
- Pour **ajouter une ligne**, cliquez dans la zone vide sous la dernière ligne.
- Pour **supprimer des lignes**, sélectionnez-les (cases à cocher) puis cliquez sur le bouton [suppr] « Supprimer les lignes sélectionnées ».

Commencez par saisir les données dans les tables de référence (`genres`, `auteurs`, `classes`) avant de renseigner `livres`, `personnes` et `exemplaires`.

Relations entre espaces

Dans le panel **Champs**, choisissez le type **Relation** pour les champs `_id` :

1. Choisissez l'espace **cible** dans la liste déroulante.
2. Le champ stockera l'identifiant de l'enregistrement lié.

Voici les relations à créer pour la bibliothèque :

Champ (espace source)	Espace cible
`livres.auteur_id`	`auteurs`
`livres.genre_id`	`genres`
`exemplaires.livre_id`	`livres`
`emprunts.exemplaire_id`	`exemplaires`
`emprunts.personne_id`	`personnes`
`personnes.classe_id`	`classes`
`coordonnees.personne_id`	`personnes`
`livres_motscles.livre_id`	`livres`
`livres_motscles.motcle_id`	`motscles`
`livres_etiquettes.livre_id`	`livres`
`livres_etiquettes.etiquette_id`	`etiquettes`

Les relations permettent ensuite de construire des vues personnalisées avec filtrage automatique (dépendances inter-widgets).

Vues personnalisées (YAML)

Les vues permettent de composer des tableaux de bord multi-espaces.

Créer une vue

1. Dans la section **Vues**, cliquez sur `+`.
2. Donnez un nom à la vue (ex. `Gestion des prêts`).
3. L'éditeur YAML s'ouvre automatiquement.

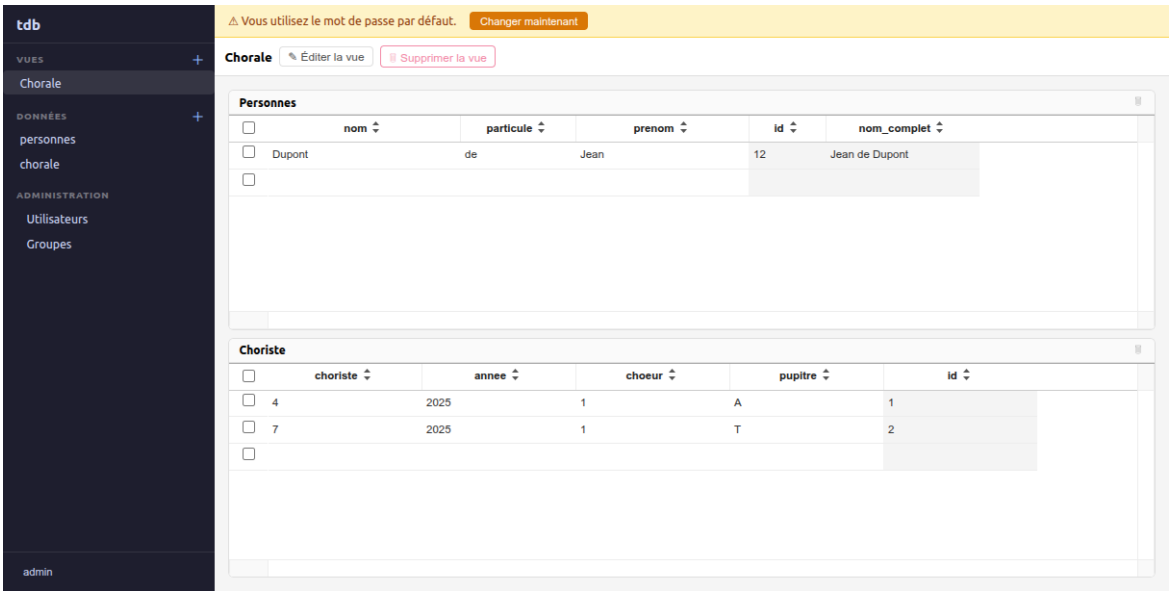

FIGURE 6 – Vue personnalisée

Éditeur YAML avec schéma ERD

Cliquer sur « **Éditer** » ouvre un modal plein écran avec deux panneaux :

- **Gauche** : éditeur CodeMirror (coloration syntaxique YAML, thème monokai).
- **Droite** : diagramme ERD interactif montrant tous les espaces et leurs relations.

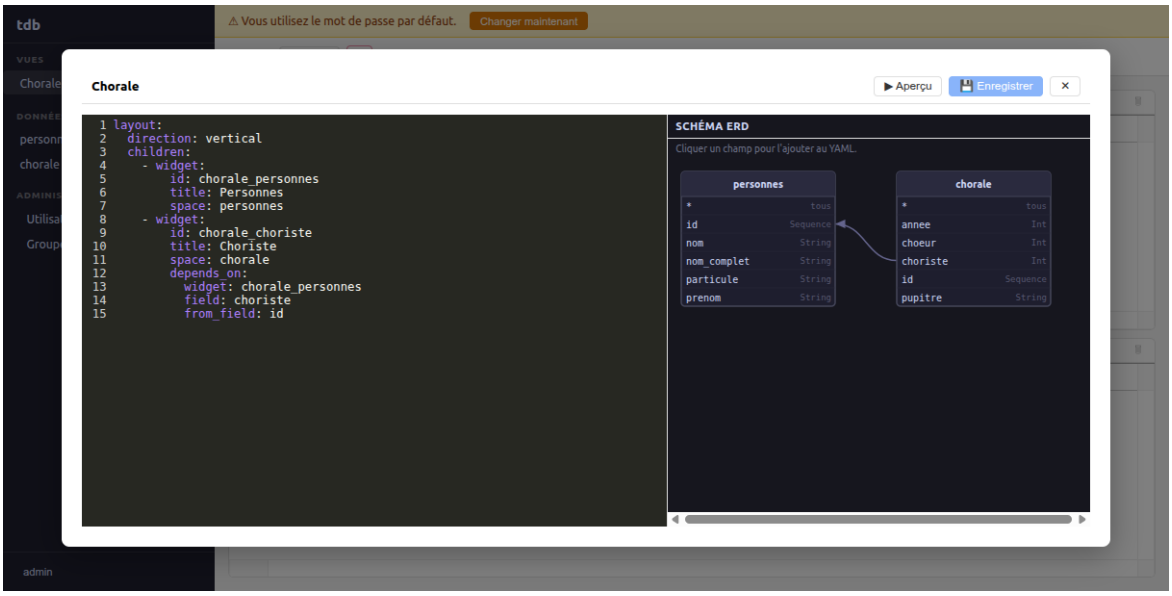

FIGURE 7 – Modal éditeur YAML et diagramme ERD

Utiliser le diagramme ERD pour construire le YAML :

Chaque boîte représente un espace. Pour chaque champ (rangée) :

- Cliquer sur ***** (première rangée, en italique) ajoute l'espace **sans restriction de colonnes**. La boîte s'illumine avec le badge * ✓.

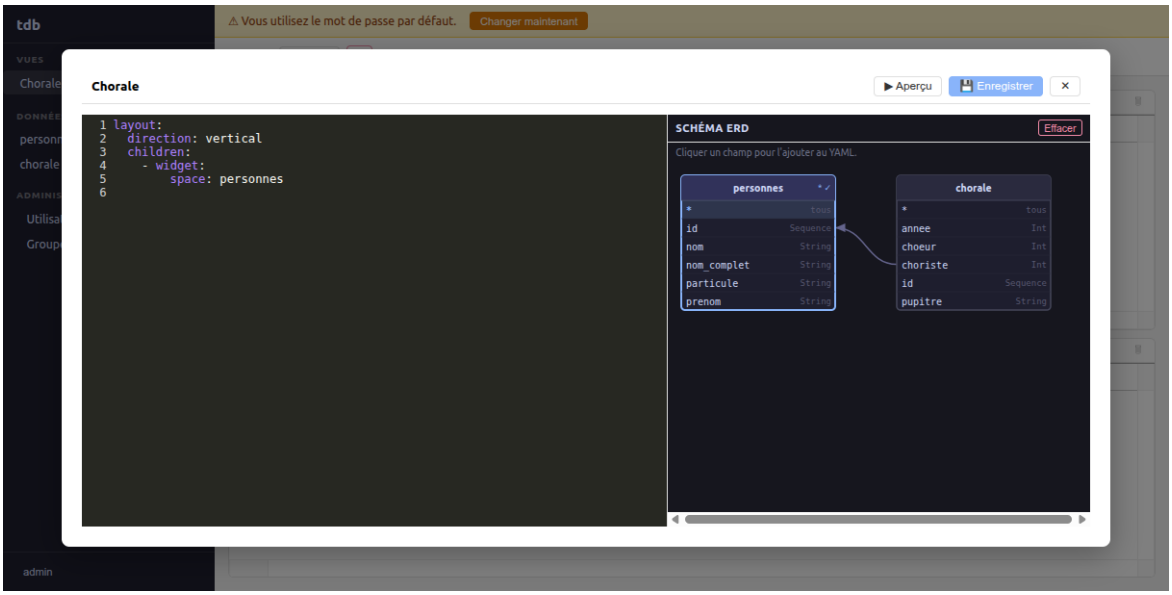

FIGURE 8 – Sélection avec le pseudo-champ *

— Cliquer sur un **champ nommé** l'ajoute à la liste `columns` du widget. Si l'espace n'est pas encore dans le YAML, il est créé. Si cet espace a une clé étrangère vers un espace déjà présent dans le YAML, un `depends_on` est généré automatiquement.

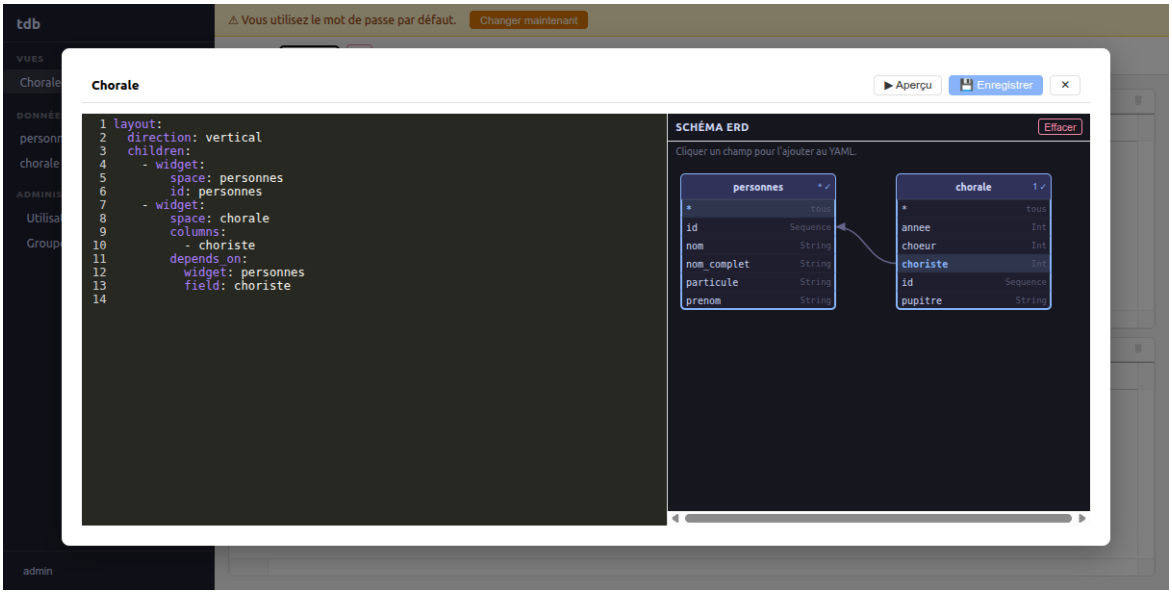

FIGURE 9 – Détection automatique de depends_on

— Recliquer un champ déjà sélectionné le retire. Retirer tous les champs supprime le widget.
— Le bouton **Effacer** réinitialise la sélection.
— Les **flèches** indiquent les clés étrangères ; les **auto-relations** (ex. `parent_id → id`) sont dessinées comme une boucle sur le côté droit de la boîte.

Synchronisation bidirectionnelle : le diagramme ERD s'initialise depuis le YAML existant au moment de l'ouverture du modal — les espaces et champs déjà définis sont mis en évidence. Modifier le YAML manuellement dans CodeMirror met à jour le diagramme en temps réel. Les champs non

gérés par le constructeur (comme `title`, `aggregate`, `computed`) sont **préservés** lors des modifications via le diagramme.

Boutons du modal :

Bouton	Action
⊠ **Enregistrer**	Sauvegarde le YAML
▸ **Aperçu**	Affiche la vue rendue
⊠	Ferme le modal sans sauvegarder

Vue « Gestion des prêts »

La vue principale de la bibliothèque affiche les livres, leurs exemplaires et les emprunts en cours, avec un récapitulatif par genre.

```yaml
layout:
  direction: vertical
  children:
    - widget:
        id: livres
        title: Catalogue des livres
        space: livres
        columns: [titre, cote_auto, auteur_id, genre_id, isbn, annee]
    - layout:
        direction: horizontal
        children:
          - widget:
              id: exemplaires
              title: Exemplaires
              space: exemplaires
              depends_on:
                widget: livres
                field: livre_id
                from_field: id
              columns: [etat, disponible]
              factor: 2
          - widget:
              id: emprunts_en_cours
              title: Emprunts en cours
              space: emprunts
              depends_on:
                widget: exemplaires
                field: exemplaire_id
                from_field: id
              columns: [personne_id, date_emprunt, date_retour]
```

```
            factor: 3
```

Widget agrégat — Emprunts par genre

Un widget de type `aggregate` affiche un tableau de synthèse en lecture seule — l'équivalent d'un `GROUP BY` SQL, calculé en Lua côté serveur.

```
- widget:
    type: aggregate
    title: Emprunts par genre
    space: emprunts
    groupBy: [genre_id]
    aggregate:
      - fn: count
        as: nb_emprunts
    computed:
      - as: label
        expr: "row.genre_id + ' (' + row.nb_emprunts + ')'"
      - as: statut
        expr: "row.nb_emprunts > 10 ? 'Actif' : 'Peu emprunté'"
```

Les fonctions disponibles sont `sum`, `count`, `avg`, `min`, `max`. L'alias `as` est optionnel (nom par défaut : `fn_champ`, ex. `count`).

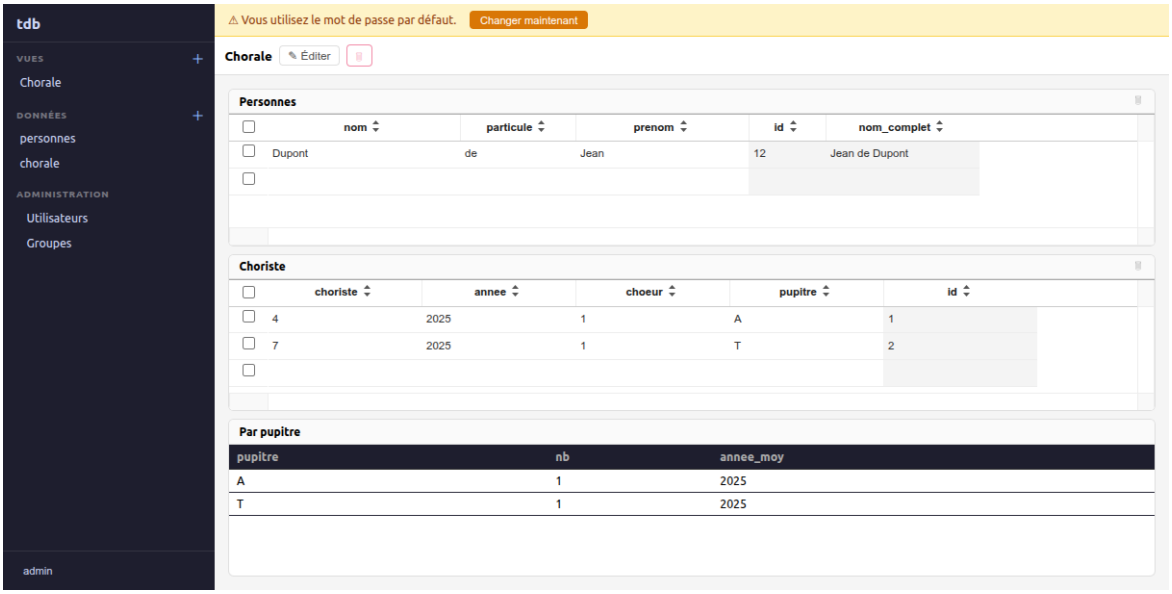

FIGURE 10 – Exemple de widget agrégat

Gestion des utilisateurs et droits (admin)

La section **Administration** n'est visible que pour les membres du groupe `admin`.

Utilisateurs

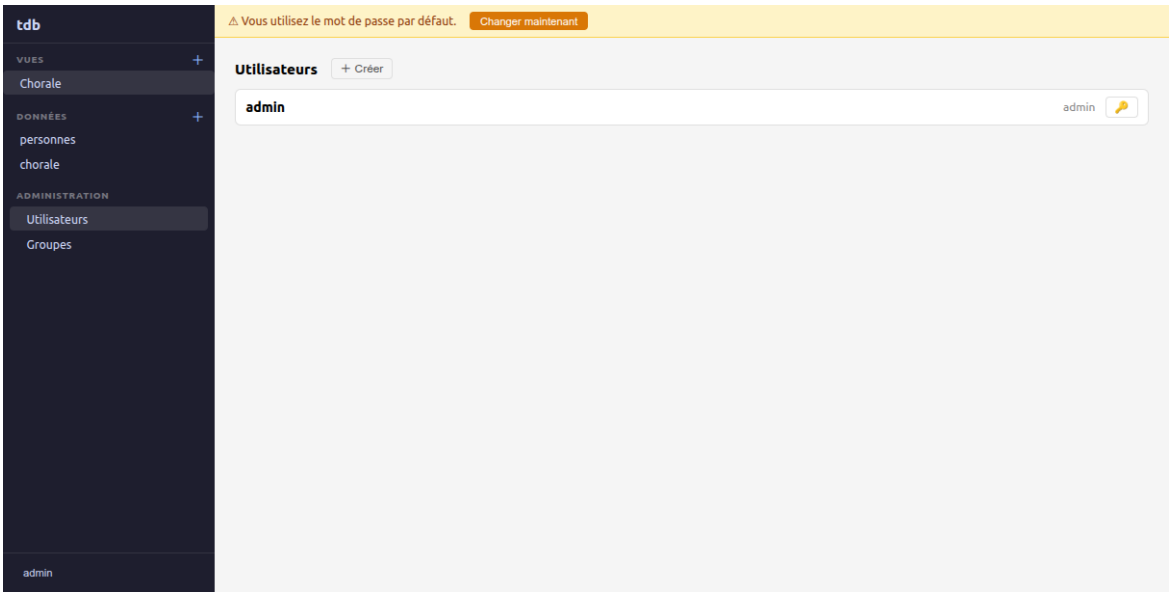

FIGURE 11 – Panel utilisateurs

- **+ Créer** : ouvre un formulaire (nom d'utilisateur, email optionnel, mot de passe).
- **[clé]** sur chaque ligne : permet à l'admin de forcer le changement de mot de passe d'un utilisateur sans connaître son mot de passe actuel.

Groupes

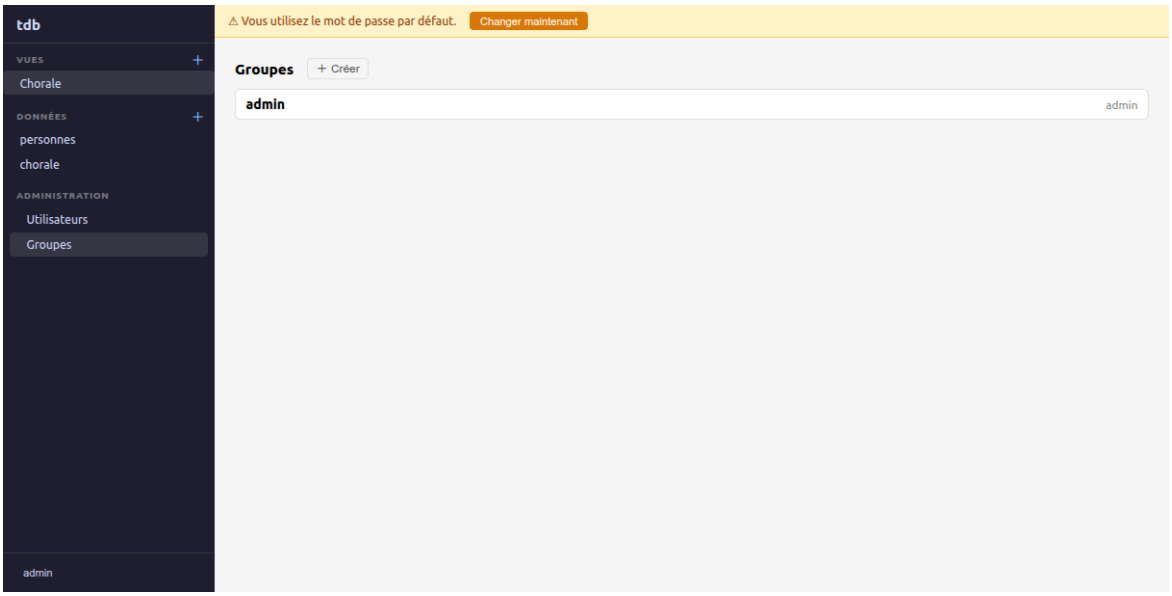

FIGURE 12 – Panel groupes

- **+ Créer** : crée un nouveau groupe.
- **[suppr]** : supprime le groupe (hors groupe `admin` qui est protégé).

Les droits (`grant` / `revoke`) sont actuellement gérables via l'API GraphQL.

Export et import de snapshots

La section **Export / Import** permet de sauvegarder et restaurer une application tdb.

Exporter :

- **Structure seule** : télécharge un fichier `.tdb.yaml` contenant le schéma complet (espaces, champs, relations, vues, groupes, permissions) sans les données.
- **Structure + données** : inclut également toutes les lignes de chaque espace.

Importer :

1. Cliquer sur **Choisir un fichier** et sélectionner un fichier `.tdb.yaml`.
2. Un **diff** s'affiche (vert = créer, rouge = supprimer, orange = modifier).
3. Choisir le mode :
   - **Fusion** : crée les éléments manquants, laisse l'existant intact.
   - **Remplacement** : supprime tout puis recrée depuis le snapshot (attention : données perdues).
4. Cliquer sur **Confirmer l'import**.

Profil utilisateur

Cliquez sur votre **nom d'utilisateur** (en bas à gauche) pour ouvrir le menu :

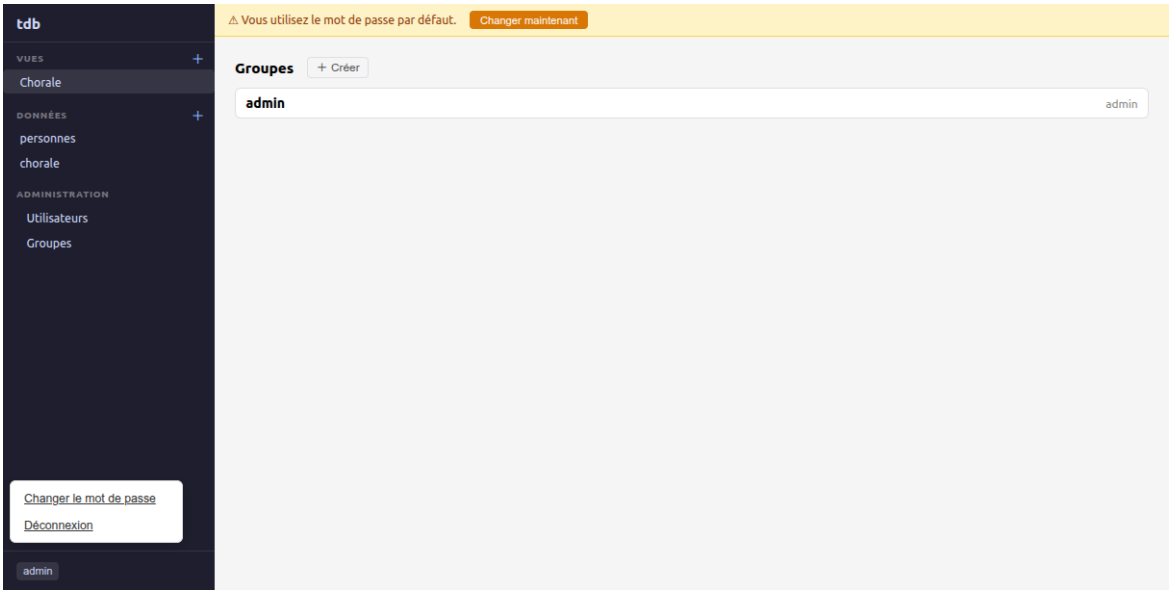

FIGURE 13 – Menu profil

- **Changer le mot de passe** : saisir l'ancien puis le nouveau mot de passe.
- **Déconnexion** : invalide la session côté serveur et revient à l'écran de connexion.

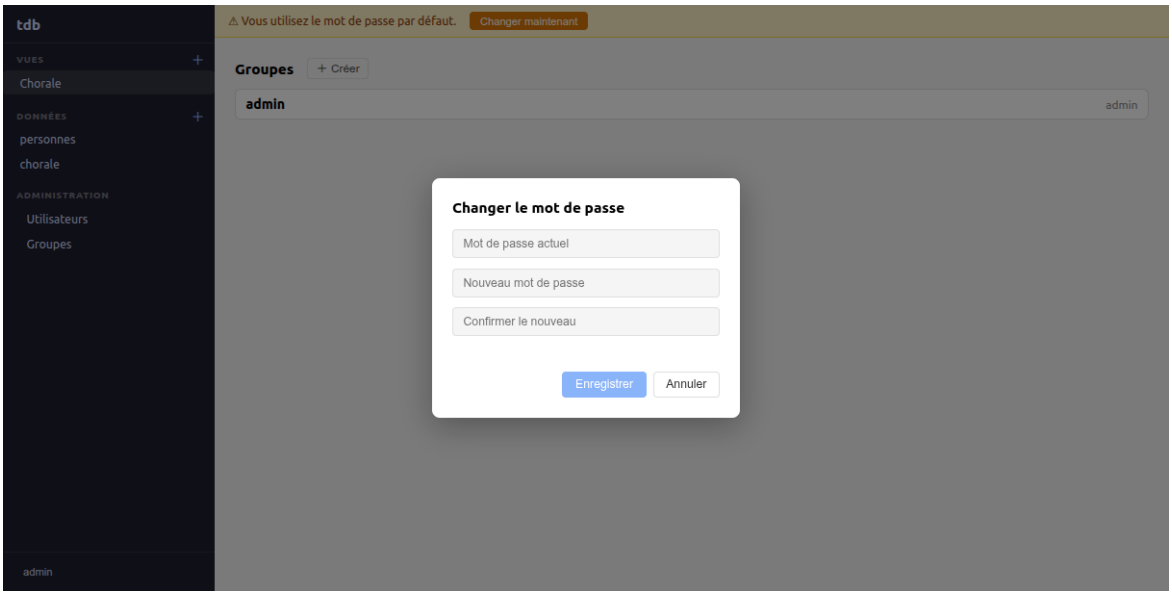

FIGURE 14 – Dialog changement de mot de passe

Renommer ou supprimer un espace

Dans la barre d'outils de l'espace :

- **(crayon)** (icône crayon) : renommer l'espace.
- **[suppr]** (icône poubelle à côté du titre) : supprimer l'espace et toutes ses données.

Raccourcis clavier

Touche	Action
Entrée	Valider la saisie dans une cellule
Échap	Annuler la saisie en cours
Tab	Passer à la cellule suivante
Entrée sur l'écran de connexion	Se connecter